RECOMMENDATION ENGINES

- Till recently, people generally tended to buy products recommended to them by their friends or the people they trust.
- This used to be the primary method of purchase when there was any doubt about the product.
- But with the advent of the digital age, that circle has expanded to include online sites that utilize some sort of recommendation engine.
- **A recommendation engine filters the data using different algorithms and recommends the most relevant items to users. It first captures the past behaviour of a customer and based on that, recommends products which the users might be likely to buy.**

- If a completely new user visits an e-commerce site, that site will not have any past history of that user.
- So how does the site go about recommending products to the user in such a scenario?
- One possible solution could be to recommend the best selling products, i.e. the products which are high in demand.
- Another possible solution could be to recommend the products which would bring the maximum profit to the business.
- If we can recommend a few items to a customer based on their needs and interests, it will create a positive impact on the user experience and lead to frequent visits.
- Hence, businesses nowadays are building smart and intelligent recommendation engines by studying the past behaviour of their users.

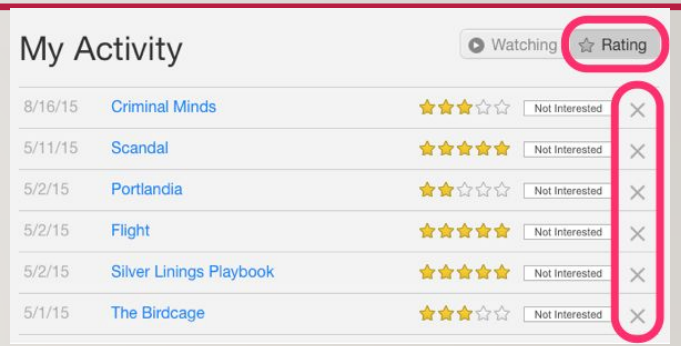
HOW DOES A RECOMMENDATION ENGINE WORK?

- Before we deep dive into this topic, first we'll think of how we can recommend items to users:
  - We can recommend items to a user which are most popular among all the users
  - We can divide the users into multiple segments based on their preferences (user features) and recommend items to them based on the segment they belong to
- Both of the above methods have their drawbacks. In the first case, the most popular items would be the same for each user so everybody will see the same recommendations.
- While in the second case, as the number of users increases, the number of features will also increase. So classifying the users into various segments will be a very difficult task.

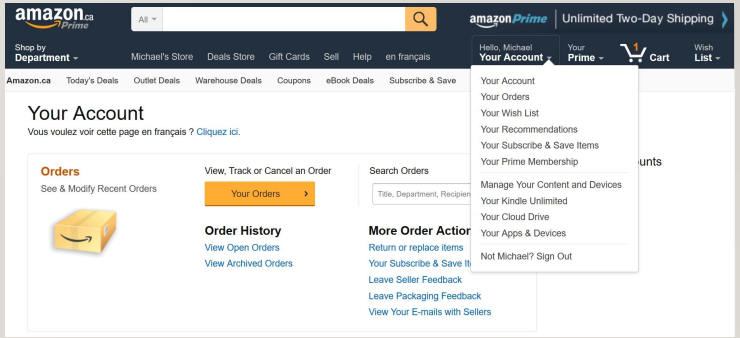
- The main problem here is that we are unable to tailor recommendations based on the specific interest of the users.
- It's like Amazon is recommending you buy a laptop just because it's been bought by the majority of the shoppers.
- But thankfully, Amazon (or any other big firm) does not recommend products using the above mentioned approach.
- They use some personalized methods which help them in recommending products more accurately.
- Let's now focus on how a recommendation engine works by going through the following steps.

DATA COLLECTION

- This is the first and most crucial step for building a recommendation engine.
- The data can be collected by two means: explicitly and implicitly.
- Explicit data is information that is provided intentionally, i.e. input from the users such as movie ratings.
- Implicit data is information that is not provided intentionally but gathered from available data streams like search history, clicks, order history, etc.



In the above image, Netflix is collecting the data explicitly in the form of ratings given by user to different movies.



Here the order history of a user is recorded by Amazon which is an example of implicit mode of data collection.

DATA STORAGE

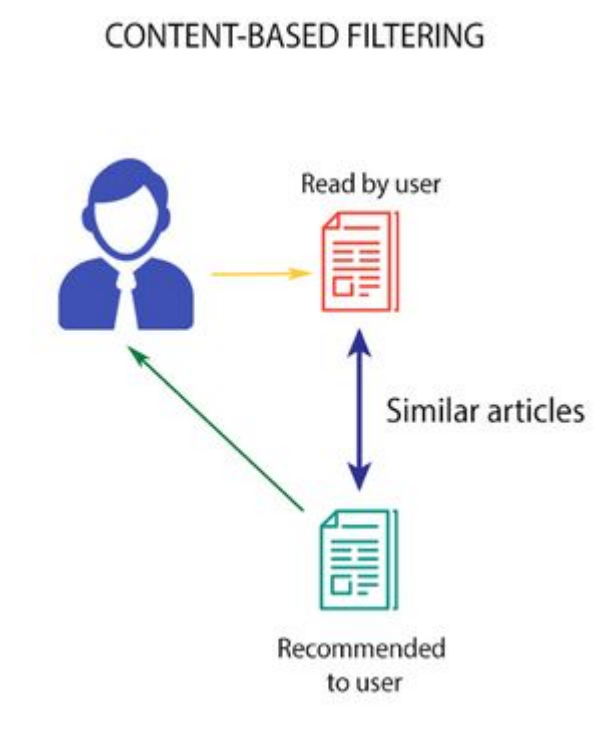
- The amount of data dictates how good the recommendations of the model can get.
- For example, in a movie recommendation system, the more ratings users give to movies, the better the recommendations get for other users.
- The type of data plays an important role in deciding the type of storage that has to be used.
- This type of storage could include a standard SQL database, a NoSQL database or some kind of object storage

FILTERING THE DATA

- After collecting and storing the data, we have to filter it so as to extract the relevant information required to make the final recommendations.
- There are various algorithms that help us make the filtering process easier. In the next section, we will go through each algorithm in detail.

CONTENT BASED FILTERING

- This algorithm recommends products which are similar to the ones that a user has liked in the past.
- For example, if a person has liked the movie “Inception”, then this algorithm will recommend movies that fall under the same genre.
- But how does the algorithm understand which genre to pick and recommend movies from?



- **Consider the example of Netflix.**
- They save all the information related to each user in a vector form.
- This vector contains the past behaviour of the user, i.e. the movies liked/disliked by the user and the ratings given by them.
- This vector is known as the *profile vector*.
- All the information related to movies is stored in another vector called the *item vector*.
- Item vector contains the details of each movie, like genre, cast, director, etc.

- The content-based filtering algorithm finds the cosine of the angle between the profile vector and item vector, i.e. **cosine similarity**. Suppose A is the profile vector and B is the item vector, then the similarity between them can be calculated as:

$$sim(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$

- Based on the cosine value, which ranges between -1 to 1, the movies are arranged in descending order and one of the two below approaches is used for recommendations:
- **Top-n approach**: where the top n movies are recommended (Here n can be decided by the business)
- **Rating scale approach**: Where a threshold is set and all the movies above that threshold are recommended

- Other methods that can be used to calculate the similarity are:

- **Euclidean Distance**: Similar items will lie in close proximity to each other if plotted in n-dimensional space. So, we can calculate the distance between items and based on that distance, recommend items to the user. The formula for the euclidean distance is given by:

$$\text{Euclidean Distance} = \sqrt{(x_1 - y_1)^2 + \ldots + (x_N - y_N)^2}$$

- **Pearson's Correlation**: It tells us how much two items are correlated. Higher the correlation, more will be the similarity. Pearson's correlation can be calculated using the following formula:

$$sim(u,v) = \frac{\sum(r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum(r_{ui} - \bar{r}_u)^2}\sqrt{\sum(r_{vi} - \bar{r}_v)^2}}$$

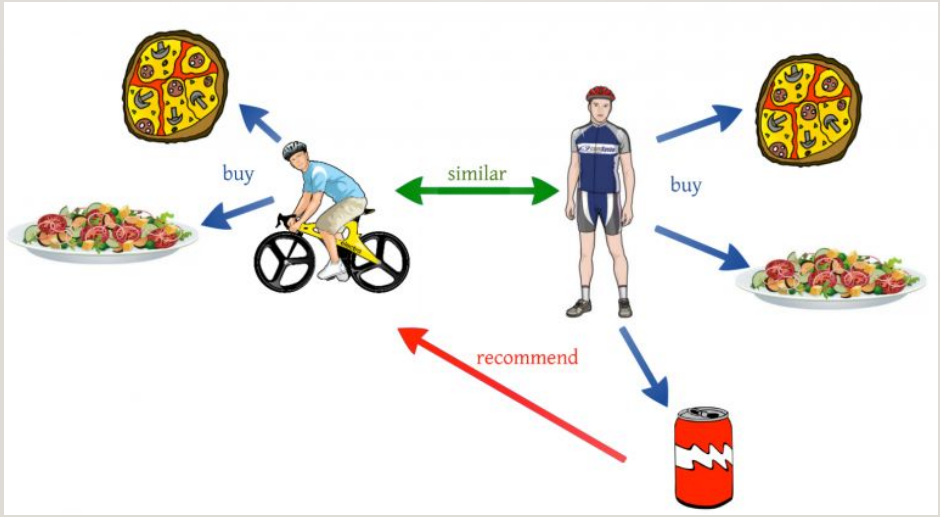
- A major drawback of this algorithm is that it is limited to recommending items that are of the same type.
- It will never recommend products which the user has not bought or liked in the past.
- So if a user has watched or liked only action movies in the past, the system will recommend only action movies. It's a very narrow way of building an engine.
- To improve on this type of system, we need an algorithm that can recommend items not just based on the content, but the behaviour of users as well.

COLLABORATIVE FILTERING

- Let us understand this with an example.
- If person A likes 3 movies, say Interstellar, Inception and Predestination, and person B likes Inception, Predestination and The Prestige, then they have almost similar interests.
- We can say with some certainty that A should like The Prestige and B should like Interstellar.
- The collaborative filtering algorithm uses “User Behavior” for recommending items.
- This is one of the most commonly used algorithms in the industry as it is not dependent on any additional information.
- There are different types of collaborating filtering techniques and we shall look at them in detail on next slides.

USER-USER COLLABORATIVE FILTERING

- This algorithm first finds the similarity score between users.
- Based on this similarity score, it then picks out the most similar users and recommends products which these similar users have liked or bought previously.



- In terms of our movies example from earlier, this algorithm finds the similarity between each user based on the ratings they have previously given to different movies. T
- he prediction of an item for a user *u* is calculated by computing the weighted sum of the user ratings given by other users to an item *i*.
- The prediction *Pu,i* is given by:

$$P_{u,i} = \frac{\sum_v (r_{v,i} * s_{u,v})}{\sum_v s_{u,v}}$$

- Here,
- *Pu,i* is the prediction of an item
- *Rv,i* is the rating given by a user *v* to a movie *i*
- *Su,v* is the similarity between users

- Now, we have the ratings for users in profile vector and based on that we have to predict the ratings for other users. Following steps are followed to do so:
- For predictions we need the similarity between the user u and v. We can make use of Pearson correlation.
- First we find the items rated by both the users and based on the ratings, correlation between the users is calculated.
- The predictions can be calculated using the similarity values. This algorithm, first of all calculates the similarity between each user and then based on each similarity calculates the predictions. **Users having higher correlation will tend to be similar.**
- Based on these prediction values, recommendations are made.

- Let us understand it with an example:
- Consider the user-movie rating matrix:

User/Movie	x1	x2	x3	x4	x5	Mean User Rating
A	4	1	–	4	–	3
B	–	4	–	2	3	3
C	–	1	–	4	4	3

- Here we have a user movie rating matrix.
- To understand this in a more practical manner, let's find the similarity between users (A, C) and (B, C) in the above table.
- Common movies rated by A and C are movies x2 and x4 and by B and C are movies x2, x4 and x5.

$$r_{AC} = [(1-3)*(1-3) + (4-3)*(4-3)]/[((1-3)^2 + (4-3)^2)^{1/2} * ((1-3)^2 + (4-3)^2)^{1/2}] = 1$$

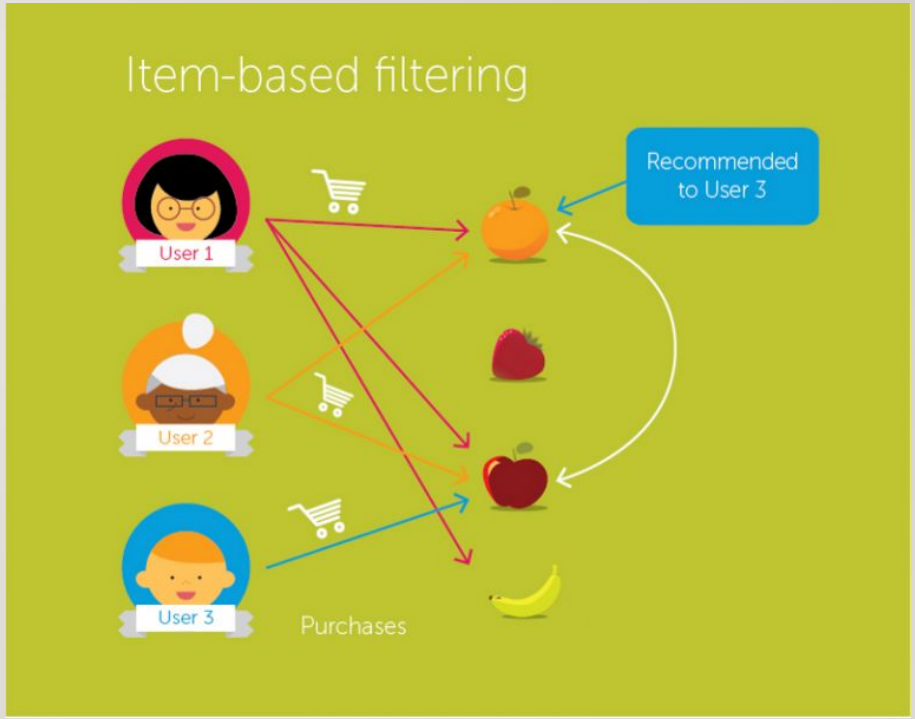
$$r_{BC} = [(4-3)*(1-3) + (2-3)*(4-3) + (3-3)*(4-3)]/[((4-3)^2 + (2-3)^2 + (3-3)^2)^{1/2} * ((1-3)^2+(4-3)^2 + (4-3)^2)^{1/2}] = -0.866$$

- The correlation between user A and C is more than the correlation between B and C.
- Hence users A and C have more similarity and the movies liked by user A will be recommended to user C and vice versa.
- This algorithm is quite time consuming as it involves calculating the similarity for each user and then calculating prediction for each similarity score.
- One way of handling this problem is to select only a few users (neighbors) instead of all to make predictions, i.e. instead of making predictions for all similarity values, we choose only few similarity values.

- There are various ways to select the neighbours:
  - Select a threshold similarity and choose all the users above that value
  - Randomly select the users
  - Arrange the neighbours in descending order of their similarity value and choose top-N users
  - Use clustering for choosing neighbours
- This algorithm is useful when the number of users is less. Its not effective when there are a large number of users as it will take a lot of time to compute the similarity between all user pairs.
- This leads us to item-item collaborative filtering, which is effective when the number of users is more than the items being recommended.

ITEM-ITEM COLLABORATIVE FILTERING

- In this algorithm, we compute the similarity between each pair of items.
- So in our case we will find the similarity between each movie pair and based on that, we will recommend similar movies which are liked by the users in the past.
- This algorithm works similar to user-user collaborative filtering with just a little change – instead of taking the weighted sum of ratings of "user-neighbours", we take the weighted sum of ratings of "item-neighbours".



- The prediction is given by:

$$P_{u,i} = \frac{\sum_N (s_{i,N} * R_{u,N})}{\sum_N (|s_{i,N}|)}$$

- Now we will find the similarity between items.

$$sim(i,j) = cos(\vec{i}, \vec{j}) = \frac{\vec{i} \cdot \vec{j}}{\|\vec{i}\|_2 * \|\vec{j}\|_2}$$

- Now, as we have the similarity between each movie and the ratings, predictions are made and based on those predictions, similar movies are recommended.
- Let us understand it with an example.

User/Movie	x1	x2	x3	x4	x5
A	4	1	2	4	4
B	2	4	4	2	1
C	–	1	–	3	4
Mean Item Rating	3	2	3	3	3

- Here the mean item rating is the average of all the ratings given to a particular item (compare it with the table we saw in user-user filtering).
- Instead of finding the user-user similarity as we saw earlier, we find the item-item similarity.
- To do this, first we need to find such users who have rated those items and based on the ratings, similarity between the items is calculated.
- Let us find the similarity between movies (x1, x4) and (x1, x5). Common users who have rated movies x1 and x4 are A and B while the users who have rated movies x1 and x5 are also A and B.

$$C_{14} = [(4-3)*(4-3) + (2-3)*(2-3)] / [((4-3)^2 + (2-3)^2)^{1/2} * ((4-3)^2 + (2-3)^2)^{1/2}] = 1$$

$$C_{15} = [(4-3)*(4-3) + (2-3)*(1-3)] / [((4-3)^2 + (2-3)^2)^{1/2} * ((4-3)^2 + (1-3)^2)^{1/2}] = 0.94$$

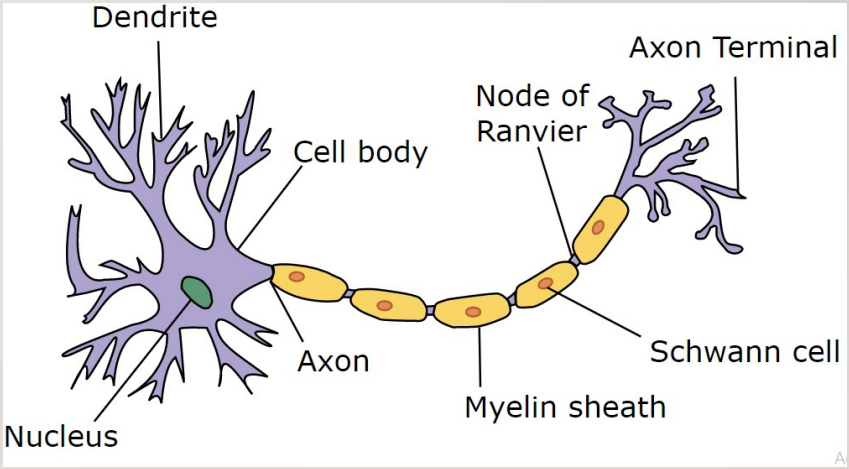
- The similarity between movie x1 and x4 is more than the similarity between movie x1 and x5.
- So based on these similarity values, if any user searches for movie x1, they will be recommended movie x4 and vice versa.
- There is a question which we must know the answer to – what will happen if a new user or a new item is added in the dataset? It is called a **Cold Start**. There can be two types of cold start:
  - Visitor Cold Start
  - Product Cold Start

- Visitor Cold Start means that a new user is introduced in the dataset.
- Since there is no history of that user, the system does not know the preferences of that user.
- It becomes harder to recommend products to that user.
- So, how can we solve this problem?
- One basic approach could be to apply a popularity based strategy, i.e. recommend the most popular products.
- These can be determined by what has been popular recently overall or regionally. Once we know the preferences of the user, recommending products will be easier.

- On the other hand, Product Cold Start means that a new product is launched in the market or added to the system.
- User action is most important to determine the value of any product.
- More the interaction a product receives, the easier it is for our model to recommend that product to the right user.
- We can make use of Content based filtering to solve this problem.
- The system first uses the content of the new product for recommendations and then eventually the user actions on that product.

WHAT IS NEURAL NETWORK?

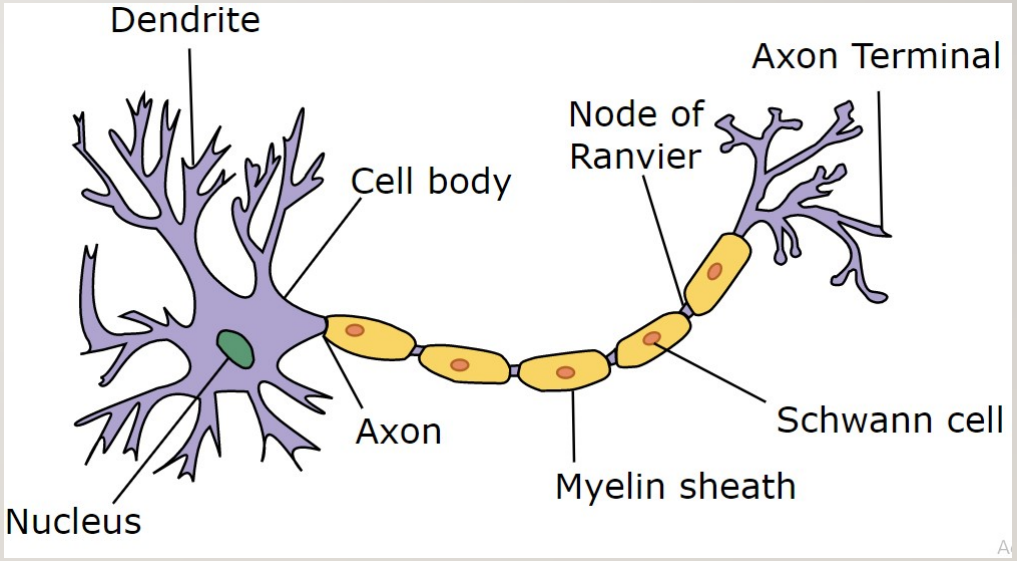
- Network means it is an interconnection of some sort between something.
- Neural means neurons.
- Neurons are the fundamental blocks of the human brain. All our life experiences, feeling, emotions, basically our entire personality is defined by those neurons. Every decision we make in your daily life, no matter how small or big are driven by those neurons.



- So neural network means the network of neurons.
- That's it. You might ask "Why are we discussing biology in neural networks?". Well in the data science realm, when we are discussing neural networks, those are basically inspired by the structure of the human brain hence the name.
- Another important thing to consider is that individual neurons themselves cannot do anything. It is the collection of neurons where the real magic happens.

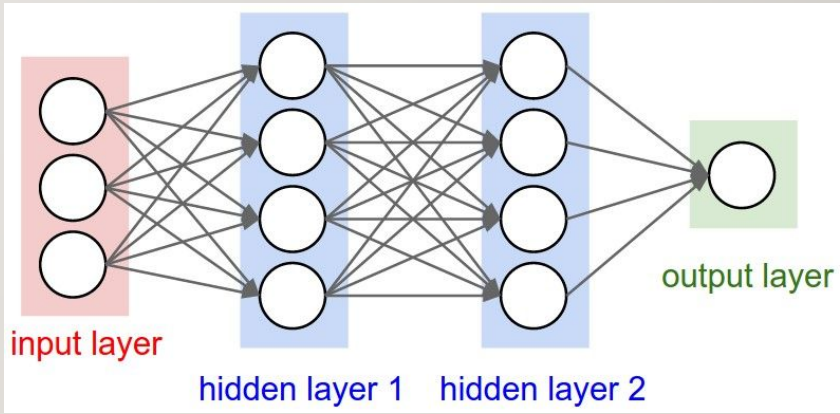
STRUCTURE OF NEURAL NETWORKS

- Since we already said that neural networks are something that is inspired by the human brain let's first understand the structure of the human brain first.
- Each neuron composed of three parts:-
- Axon
- Dendrites
- Body



- Neuron works in association with each other.
- Each neuron receives signals from another neuron and this is done by Dendrite.
- Axon is something that is responsible for transmitting output to another neuron.
- Those Dendrites and Axons are interconnected with the help of the body(simplified term).
- Now let's understand its relevance to our neural network with the one used in the data science realm.

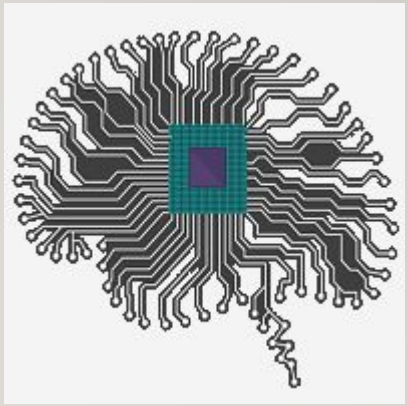
- In any neural network, there are 3 layers present:
- Input Layer: It functions similarly to that of dendrites. The purpose of this layer is to accept input from another neuron.
- Hidden Layer: These are the layers that perform the actual operation
- Output Layer: It functions similarly to that of axons. The purpose of this layer to transmit the generated output to other neurons.



- One thing to be noted here is that in the above diagram we have 2 hidden layers.
- But there is no limit on how many hidden layers should be here. It can be as low as 1 or as high as 100 or maybe even 1000!
- Now it's time to answer our question. "Why the study of neural networks called Deep Learning"? Well, the answer is right in the figure itself.
- It is because of the presence of multiple hidden layers in the neural network hence the name "Deep".
- Also after creating the neural network, we have to train it in order to solve the problem hence the name "Learning". Together these two constitute "Deep Learning"

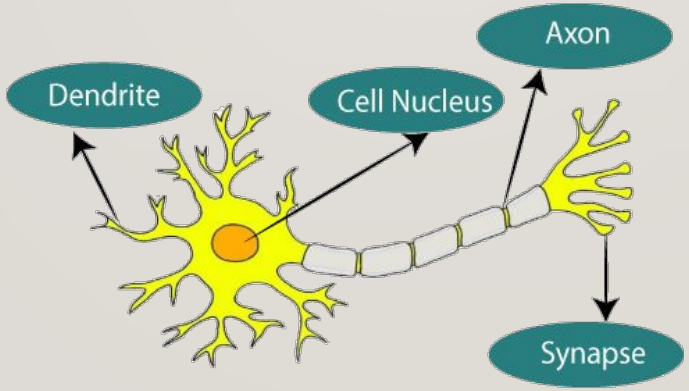
ARTIFICIAL NEURAL NETWORK

- The term "Artificial neural network" refers to a biologically inspired sub-field of artificial intelligence modelled after the brain.
- An Artificial neural network is usually a computational network based on biological neural networks that construct the structure of the human brain.
- Similar to a human brain has neurons interconnected to each other, artificial neural networks also have neurons that are linked to each other in various layers of the networks. These neurons are known as nodes.

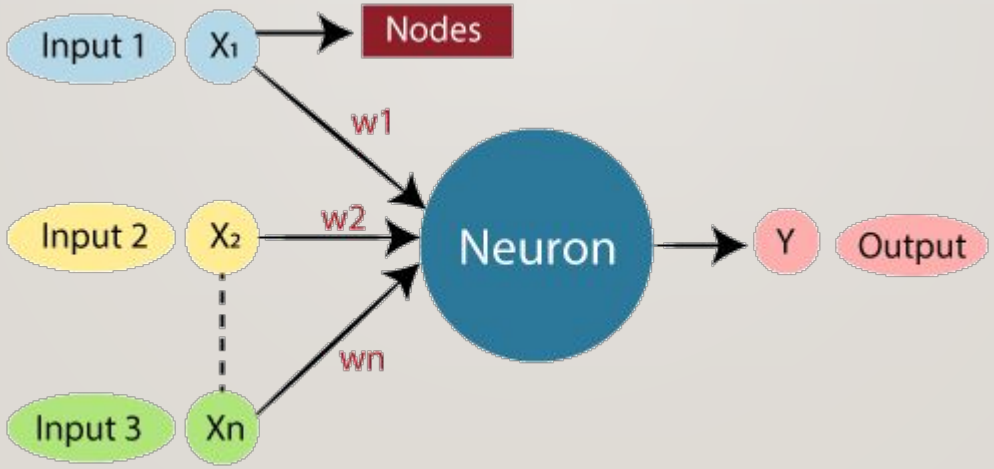


WHAT IS ARTIFICIAL NEURAL NETWORK?

- The term "**Artificial Neural Network**" is derived from Biological neural networks that develop the structure of a human brain.
- Similar to the human brain that has neurons interconnected to one another, artificial neural networks also have neurons that are interconnected to one another in various layers of the networks. These neurons are known as nodes.



- The given figure illustrates the typical diagram of Biological Neural Network.
- The typical Artificial Neural Network looks something like the given figure.



- Dendrites from Biological Neural Network represent inputs in Artificial Neural Networks, cell nucleus represents Nodes, synapse represents Weights, and Axon represents Output.
- Relationship between Biological neural network and artificial neural network:

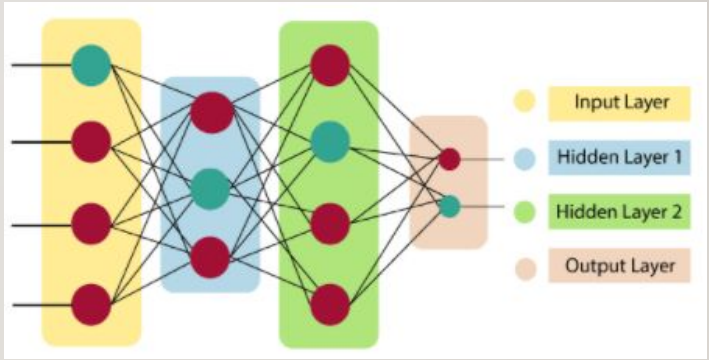
Biological Neural Network	Artificial Neural Network
Dendrites	Inputs
Cell nucleus	Nodes
Synapse	Weights
Axon	Output

- An **Artificial Neural Network** in the field of **Artificial intelligence** where it attempts to mimic the network of neurons makes up a human brain so that computers will have an option to understand things and make decisions in a human-like manner.
- The artificial neural network is designed by programming computers to behave simply like interconnected brain cells.
- There are around 1000 billion neurons in the human brain.
- Each neuron has an association point somewhere in the range of 1,000 and 100,000.
- In the human brain, data is stored in such a manner as to be distributed, and we can extract more than one piece of this data when necessary from our memory parallelly.
- can say that the human brain is made up of incredibly amazing parallel processors.

- We can understand the artificial neural network with an example, consider an example of a digital logic gate that takes an input and gives an output. "OR" gate, which takes two inputs.
- If one or both the inputs are "On," then we get "On" in output.
- If both the inputs are "Off," then we get "Off" in output.
- Here the output depends upon input.
- Our brain does not perform the same task.
- The outputs to inputs relationship keep changing because of the neurons in our brain, which are "learning."

THE ARCHITECTURE OF AN ARTIFICIAL NEURAL NETWORK:

- To understand the concept of the architecture of an artificial neural network, we have to understand what a neural network consists of.
- In order to define a neural network that consists of a large number of artificial neurons, which are termed units arranged in a sequence of layers.
- Lets us look at various types of layers available in an artificial neural network.
- Artificial Neural Network primarily consists of three layers:



- **Input Layer:** As the name suggests, it accepts inputs in several different formats provided by the programmer.
- **Hidden Layer:** The hidden layer presents in-between input and output layers. It performs all the calculations to find hidden features and patterns.
- **Output Layer:** The input goes through a series of transformations using the hidden layer, which finally results in output that is conveyed using this layer.
- The artificial neural network takes input and computes the weighted sum of the inputs and includes a bias. This computation is represented in the form of a transfer function.

$$\sum_{i=1}^{n} Wi * Xi + b$$

- It determines weighted total is passed as an input to an activation function to produce the output.
- Activation functions choose whether a node should fire or not.
- Only those who are fired make it to the output layer.
- There are distinctive activation functions available that can be applied upon the sort of task we are performing.

ADVANTAGES OF ARTIFICIAL NEURAL NETWORK (ANN)

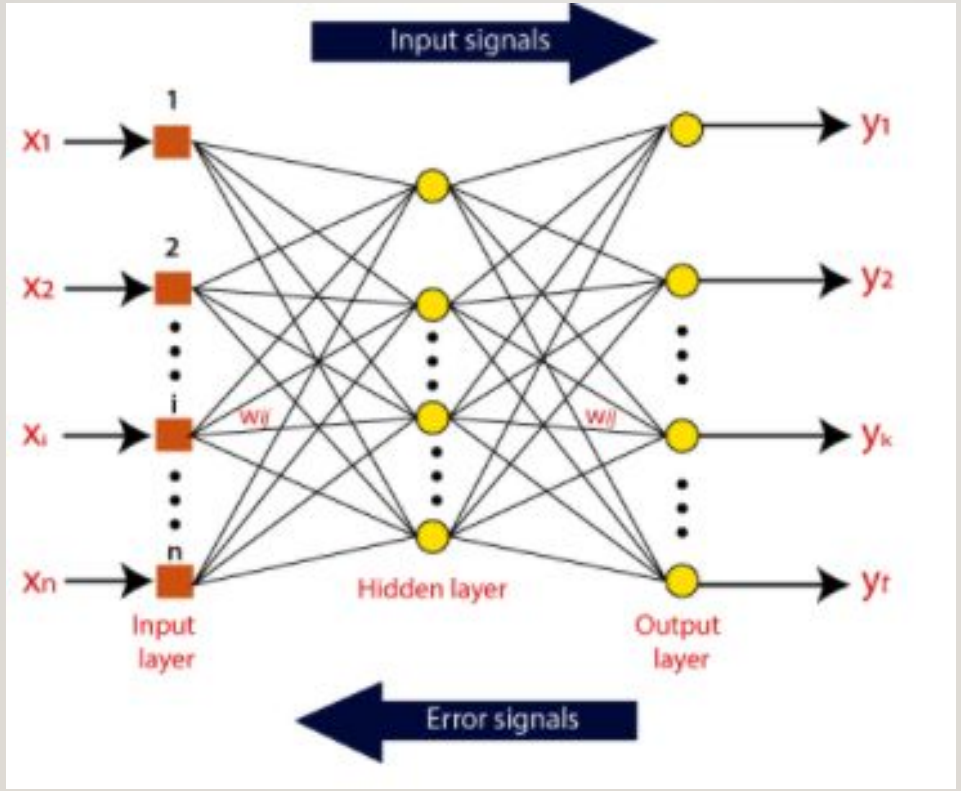
- **Parallel processing capability:** Artificial neural networks have a numerical value that can perform more than one task simultaneously.
- **Storing data on the entire network:** Data that is used in traditional programming is stored on the whole network, not on a database. The disappearance of a couple of pieces of data in one place doesn't prevent the network from working.
- **Capability to work with incomplete knowledge:** After ANN training, the information may produce output even with inadequate data. The loss of performance here relies upon the significance of missing data.
- **Having a memory distribution:** For ANN is to be able to adapt, it is important to determine the examples and to encourage the network according to the desired output by demonstrating these examples to the network. The succession of the network is directly proportional to the chosen instances, and if the event can't appear to the network in all its aspects, it can produce false output.
- **Having fault tolerance:** Extortion of one or more cells of ANN does not prohibit it from generating output, and this feature makes the network fault-tolerance.

DISADVANTAGES OF ARTIFICIAL NEURAL NETWORK:

- **Assurance of proper network structure:** There is no particular guideline for determining the structure of artificial neural networks. The appropriate network structure is accomplished through experience, trial, and error.
- **Unrecognized behaviour of the network:** It is the most significant issue of ANN. When ANN produces a testing solution, it does not provide insight concerning why and how. It decreases trust in the network.
- **Hardware dependence:** Artificial neural networks need processors with parallel processing power, as per their structure. Therefore, the realization of the equipment is dependent.
- **Difficulty of showing the issue to the network:** ANNs can work with numerical data. Problems must be converted into numerical values before being introduced to ANN. The presentation mechanism to be resolved here will directly impact the performance of the network. It relies on the user's abilities.
- **The duration of the network is unknown:** The network is reduced to a specific value of the error, and this value does not give us optimum results.

HOW DO ARTIFICIAL NEURAL NETWORKS WORK?

- Artificial Neural Network can be best represented as a weighted directed graph, where the artificial neurons form the nodes.
- The association between the neurons outputs and neuron inputs can be viewed as the directed edges with weights.
- The Artificial Neural Network receives the input signal from the external source in the form of a pattern and image in the form of a vector.
- These inputs are then mathematically assigned by the notations x(n) for every n number of inputs.


Input signals
1
2
i
n
x1
x2
xi
xn
wij
wij
y1
y2
yk
yr
Hidden layer
Input layer
Output layer
Error signals

- Afterward, each of the input is multiplied by its corresponding weights (these weights are the details utilized by the artificial neural networks to solve a specific problem).
- In general terms, these weights normally represent the strength of the interconnection between neurons inside the artificial neural network.
- All the weighted inputs are summarized inside the computing unit.
- If the weighted sum is equal to zero, then bias is added to make the output non-zero or something else to scale up to the system's response.
- Bias has the same input, and weight equals to 1.
- Here the total of weighted inputs can be in the range of 0 to positive infinity.
- Here, to keep the response in the limits of the desired value, a certain maximum value is benchmarked, and the total of weighted inputs is passed through the activation function.

- The activation function refers to the set of transfer functions used to achieve the desired output.
- There is a different kind of the activation function, but primarily either linear or non-linear sets of functions.
- Some of the commonly used sets of activation functions are the Binary, linear, and Tan hyperbolic sigmoidal activation functions.
- Let us take a look at each of them in details:
- **Binary:**
- In binary activation function, the output is either a one or a 0. Here, to accomplish this, there is a threshold value set up.
- If the net weighted input of neurons is more than 1, then the final output of the activation function is returned as one or else the output is returned as 0.

- **Sigmoidal Hyperbolic:**
- The Sigmoidal Hyperbola function is generally seen as an "**S**" shaped curve. Here the tan hyperbolic function is used to approximate output from the actual net input. The function is defined as:
- **F(x) = (1/1 + exp(-????x))**
- Where ???? is considered the Steepness parameter.

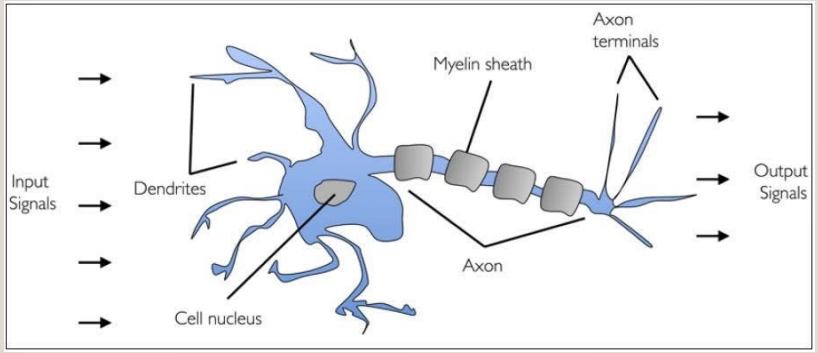
TYPES OF ARTIFICIAL NEURAL NETWORK:

- There are various types of Artificial Neural Networks (ANN) depending upon the human brain neuron and network functions, an artificial neural network similarly performs tasks.
- The majority of the artificial neural networks will have some similarities with a more complex biological partner and are very effective at their expected tasks. For example, segmentation or classification.
- **Feedback ANN:**
- In this type of ANN, the output returns into the network to accomplish the best-evolved results internally.
- As per the **University of Massachusetts**, Lowell Centre for Atmospheric Research.
- The feedback networks feed information back into itself and are well suited to solve optimization issues. The Internal system error corrections utilize feedback ANNs.

- **Feed-Forward ANN:**
- A feed-forward network is a basic neural network comprising of an input layer, an output layer, and at least one layer of a neuron.
- Through assessment of its output by reviewing its input, the intensity of the network can be noticed based on group behaviour of the associated neurons, and the output is decided.
- The primary advantage of this network is that it figures out how to evaluate and recognize input patterns.

BIOLOGICAL NEURON

- A human brain has billions of neurons. Neurons are interconnected nerve cells in the human brain that are involved in processing and transmitting chemical and electrical signals. Dendrites are branches that receive information from other neurons.



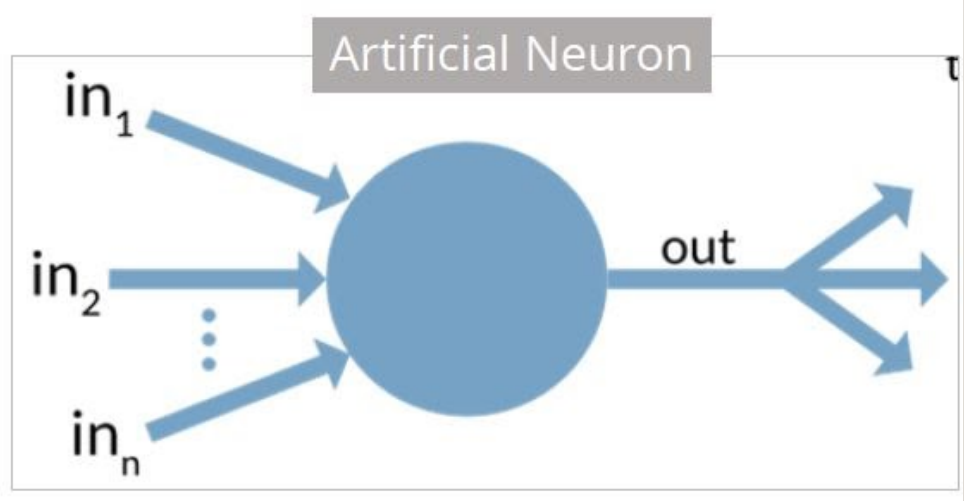
- Cell nucleus or Soma processes the information received from dendrites. Axon is a cable that is used by neurons to send information. Synapse is the connection between an axon and other neuron dendrites.

RISE OF ARTIFICIAL NEURONS

- Researchers Warren McCullock and Walter Pitts published their first concept of simplified brain cell in 1943.
- This was called McCullock-Pitts (MCP) neuron. They described such a nerve cell as a simple logic gate with binary outputs.
- Multiple signals arrive at the dendrites and are then integrated into the cell body, and, if the accumulated signal exceeds a certain threshold, an output signal is generated that will be passed on by the axon.

WHAT IS ARTIFICIAL NEURON

- An artificial neuron is a mathematical function based on a model of biological neurons, where each neuron takes inputs, weighs them separately, sums them up and passes this sum through a nonlinear function to produce output.



BIOLOGICAL NEURON VS. ARTIFICIAL NEURON

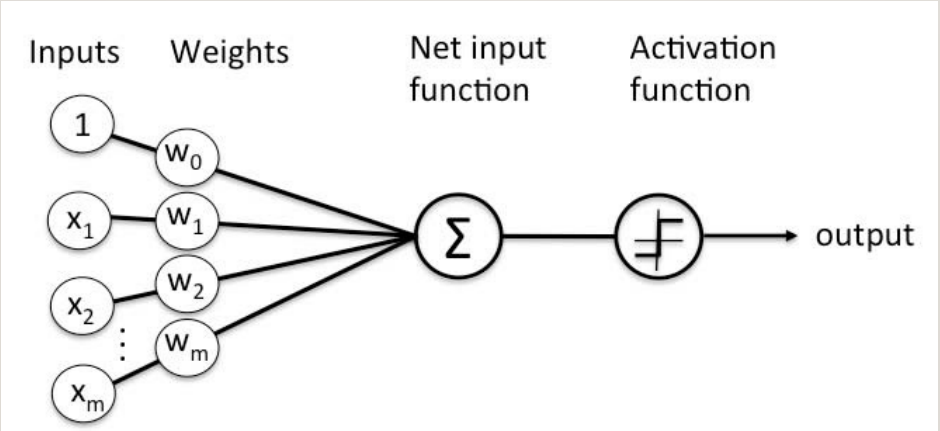
- The biological neuron is analogous to artificial neurons in the following terms:

Biological Neuron	Artificial Neuron
Cell Nucleus (Soma)	Node
Dendrites	Input
Synapse	Weights or interconnections
Axon	Output

- Artificial Neuron at a Glance
  - The artificial neuron has the following characteristics:
  - A neuron is a mathematical function modeled on the working of biological neurons
  - It is an elementary unit in an artificial neural network
  - One or more inputs are separately weighted
  - Inputs are summed and passed through a nonlinear function to produce output
  - Every neuron holds an internal state called activation signal
  - Each connection link carries information about the input signal
  - Every neuron is connected to another neuron via connection link

PERCEPTRON

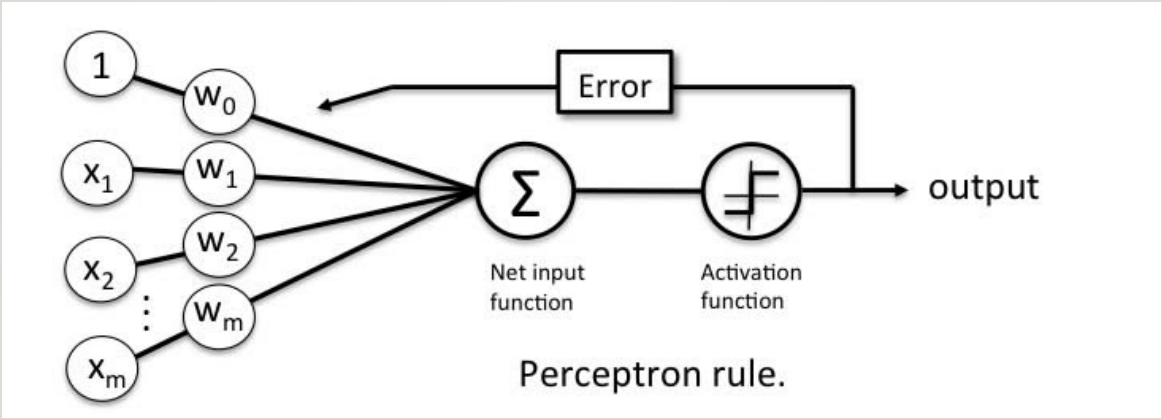
- Perceptron was introduced by Frank Rosenblatt in 1957.
- He proposed a Perceptron learning rule based on the original MCP neuron.
- A Perceptron is an algorithm for supervised learning of binary classifiers. This algorithm enables neurons to learn and processes elements in the training set one at a time.



- There are two types of Perceptrons: Single layer and Multilayer.
- Single layer - Single layer perceptrons can learn only linearly separable patterns
- Multilayer - Multilayer perceptrons or feedforward neural networks with two or more layers have the greater processing power
- The Perceptron algorithm learns the weights for the input signals in order to draw a linear decision boundary.
- This enables we to distinguish between the two linearly separable classes +1 and -1.

PERCEPTRON LEARNING RULE

- Perceptron Learning Rule states that the algorithm would automatically learn the optimal weight coefficients.
- The input features are then multiplied with these weights to determine if a neuron fires or not.



- The Perceptron receives multiple input signals, and if the sum of the input signals exceeds a certain threshold, it either outputs a signal or does not return an output.

- In the context of supervised learning and classification, this can then be used to predict the class of a sample.

PERCEPTRON FUNCTION

- Perceptron is a function that maps its input "x," which is multiplied with the learned weight coefficient; an output value "f(x)"is generated.

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

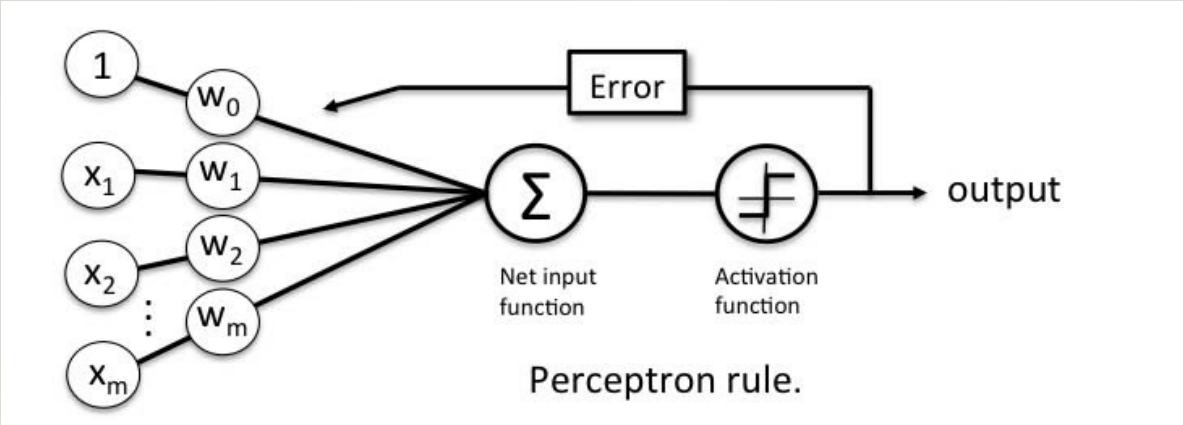
- In the equation given above:
- "w" = vector of real-valued weights
- "b" = bias (an element that adjusts the boundary away from origin without any dependence on the input value)
- "x" = vector of input x values

$$\sum_{i=1}^{m} w_i x_i$$

- "m" = number of inputs to the Perceptron
- The output can be represented as "1" or "0." It can also be represented as "1" or "-1" depending on which activation function is used.

INPUTS OF A PERCEPTRON

- A Perceptron accepts inputs, moderates them with certain weight values, then applies the transformation function to output the final result.
- The image below shows a Perceptron with a Boolean output.

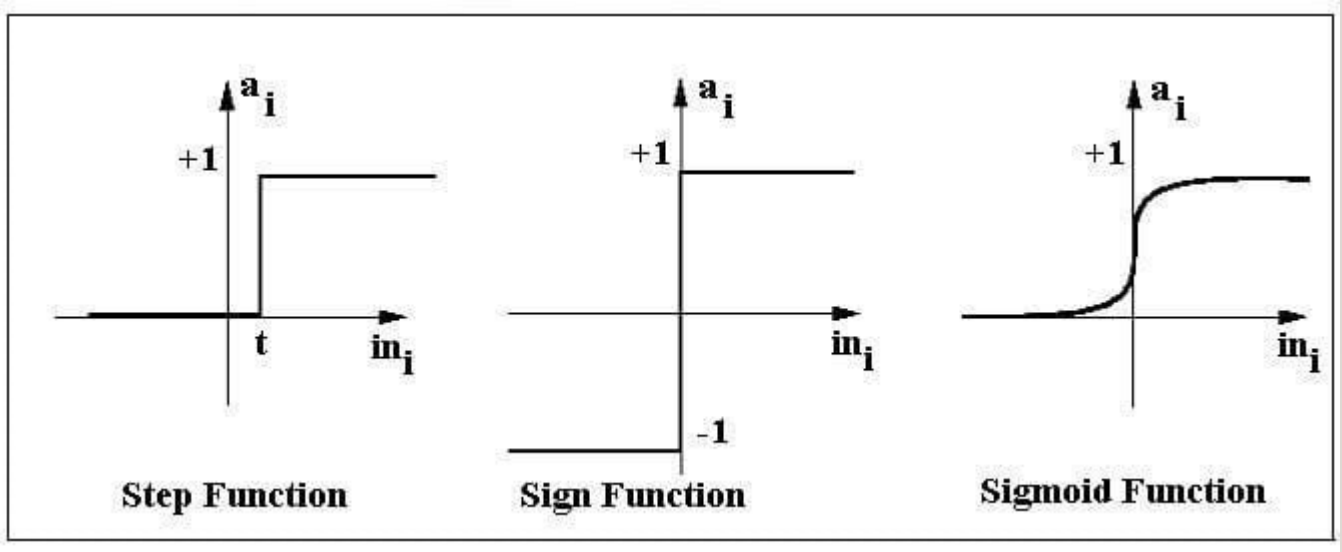


- A Boolean output is based on inputs such as salaried, married, age, past credit profile, etc.
- It has only two values: Yes and No or True and False.
- The summation function "∑" multiplies all inputs of "x" by weights "w" and then adds them up as follows:

$$w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

ACTIVATION FUNCTIONS OF PERCEPTRON

- The activation function applies a step rule (convert the numerical output into +1 or -1) to check if the output of the weighting function is greater than zero or not.



- For example:
- If $\sum$ wixi> 0 => then final output "o" = 1 (issue bank loan)
- Else, final output "o" = -1 (deny bank loan)
- Step function gets triggered above a certain value of the neuron output; else it outputs zero.
- Sign Function outputs +1 or -1 depending on whether neuron output is greater than zero or not. Sigmoid is the S-curve and outputs a value between 0 and 1.

OUTPUT OF PERCEPTRON

- Perceptron with a Boolean output:
- Inputs: x1…xn
- Output: o(x1….xn)

$$o(x_1, \ldots, x_n) = \begin{cases} 1 \text{ if } w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n > 0 \\ -1 \text{ otherwise} \end{cases}$$

- Weights: wi=> contribution of input xi to the Perceptron output;
- w0=> bias or threshold

- If $\sum w.x > 0$, output is +1, else -1.
- The neuron gets triggered only when weighted input reaches a certain threshold value.

$$o(\vec{x}) = sgn(\vec{w} \cdot \vec{x})$$

$$sgn(y) = \begin{cases} 1 \text{ if } y > 0 \\ -1 \text{ otherwise} \end{cases}$$

- An output of +1 specifies that the neuron is triggered. An output of -1 specifies that the neuron did not get triggered.
- "sgn" stands for sign function with output +1 or -1.

PERCEPTRON: DECISION FUNCTION

- A decision function $\phi(z)$ of Perceptron is defined to take a linear combination of x and w vectors.

$$\boldsymbol{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

- The value z in the decision function is given by:

$$z = w_1 x_1 + \ldots + w_m x_m$$

- The decision function is +1 if z is greater than a threshold θ, and it is -1 otherwise.

$$\phi(z) = \begin{cases} 1 & if\ z \geq \theta \\ -1 & otherwise \end{cases}$$

- This is the Perceptron algorithm.

BIAS UNIT

- For simplicity, the threshold θ can be brought to the left and represented as w0x0, where w0= -θ and x0= 1.

$$z = w_0 x_0 + w_1 x_1 + \ldots + w_m x_m = \boldsymbol{w}^T \boldsymbol{x}$$

- The value w0 is called the bias unit.
- The decision function then becomes:

$$\phi(z) = \begin{cases} 1 & if\ z \geq 0 \\ -1 & otherwise \end{cases}$$

EXAMPLE

- Suppose that we are going to work on AND Gate problem. The gate returns if and only if both inputs are true.

X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1

We are going to set weights randomly. Let's say that w1 = 0.9 and w2 = 0.9

- **Round 1**
- We will apply 1st instance to the perceptron. x1 = 0 and x2 = 0.
- Sum unit will be 0 as calculated below
- $\Sigma = x1 * w1 + x2 * w2 = 0 * 0.9 + 0 * 0.9 = 0$
- Activation unit checks sum unit is greater than a threshold. If this rule is satisfied, then it is fired and the unit will return 1, otherwise it will return 0. BTW, modern neural networks architectures do not use this kind of a step function as activation.

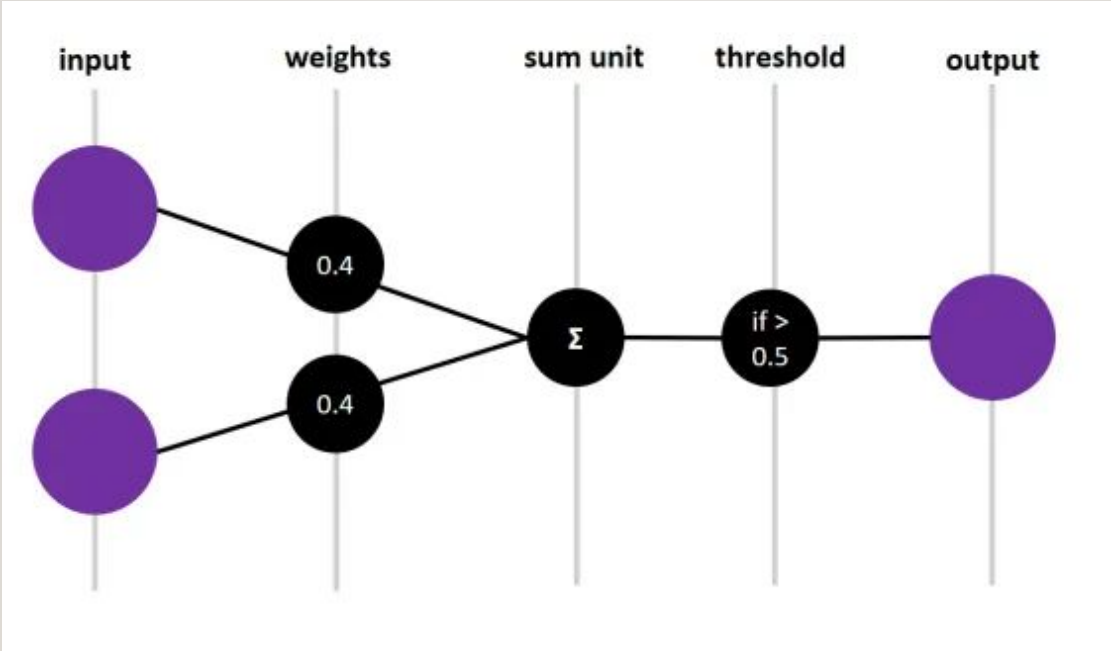
- Activation threshold would be 0.5.
- Sum unit was 0 for the 1st instance. So, activation unit would return 0 because it is less than 0.5. Similarly, its output should be 0 as well. We will not update weights because there is no error in this case.
- Let's focus on the 2nd instance. x1 = 0 and x2 = 1.
- Sum unit: $\Sigma = x1 * w1 + x2 * w2 = 0 * 0.9 + 1 * 0.9 = 0.9$

- What about errors?
- Activation unit will return 1 because sum unit is greater than 0.5. However, output of this instance should be 0. This instance is not predicted correctly. That's why, we will update weights based on the error.
- ε = actual – prediction = 0 – 1 = -1
- We will add error times learning rate value to the weights. Learning rate would be 0.5. BTW, we mostly set learning rate value between 0 and 1.
- $w1 = w1 + \alpha * \varepsilon = 0.9 + 0.5 * (-1) = 0.9 - 0.5 = 0.4$
- $w2 = w2 + \alpha * \varepsilon = 0.9 + 0.5 * (-1) = 0.9 - 0.5 = 0.4$

- Focus on the 3rd instance. x1 = 1 and x2 = 0.
- Sum unit: Σ = x1 * w1 + x2 * w2 = 1 * 0.4 + 0 * 0.4 = 0.4
- Activation unit will return 0 this time because output of the sum unit is 0.5 and it is less than 0.5. We will not update weights.
- Mention the 4rd instance. x1 = 1 and x2 = 1.
- Sum unit: Σ = x1 * w1 + x2 * w2 = 1 * 0.4 + 1 * 0.4 = 0.8
- Activation unit will return 1 because output of the sum unit is 0.8 and it is greater than the threshold value 0.5. Its actual value should 1 as well. This means that 4th instance is predicted correctly. We will not update anything.

- **Round 2**
- In previous round, we've used previous weight values for the 1st instance and it was classified correctly. Let's apply feed forward for the new weight values.
- Remember the 1st instance. x1 = 0 and x2 = 0.
- Sum unit: $\Sigma = x1 * w1 + x2 * w2 = 0 * 0.4 + 0 * 0.4 = 0.4$
- Activation unit will return 0 because sum unit is 0.4 and it is less than the threshold value 0.5. The output of the 1st instance should be 0 as well. This means that the instance is classified correctly. We will not update weights.

- Feed forward for the 2nd instance. x1 = 0 and x2 = 1.
- Sum unit: $\Sigma = x1 * w1 + x2 * w2 = 0 * 0.4 + 1 * 0.4 = 0.4$
- Activation unit will return 0 because sum unit is less than the threshold 0.5. Its output should be 0 as well. This means that it is classified correctly and we will not update weights.
- We've applied feed forward calculation for 3rd and 4th instances already for the current weight values in the previous round. They were classified correctly.


input
weights
sum unit
threshold
output
0.4
0.4
Σ
if >
0.5

THE FULL PERCEPTRON ALGORITHM IN PSEUDOCODE

$w = 0$

for i=1 to NumberOfIterations:

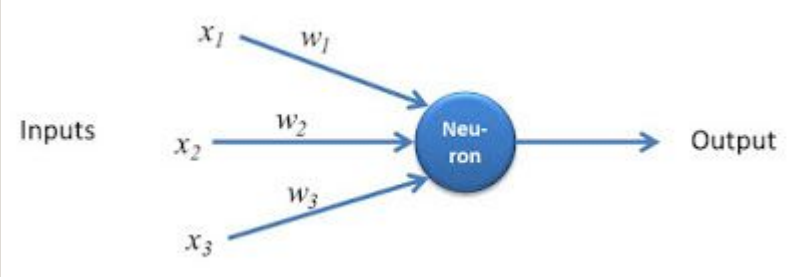
    for each data point (x, y):

        if $y(x \cdot w) \leq 0$:

            $w = w + yx$

MULTILAYER PERCEPTRON

- We already know that the basic unit of a neural network is a network that has just a single node, and this is referred to as the **perceptron**.
- The perceptron is made up of inputs x_1, x_2, …, x_n their corresponding weights w_1, w_2, …, w_n.
- A function known as activation function takes these inputs, multiplies them with their corresponding weights and produces an output y.



WHAT IS MULTILAYER PERCEPTRON?

- A multilayer perceptron is a class of neural network that is made up of at least 3 nodes. So now we can see the difference.
- Also, each of the node of the multilayer perceptron, except the input node is a neuron that uses a *non-linear activation function*.
- The nodes of the multilayer perceptron are arranged in layers.
- The input layer
- The output layer
- Hidden layers: layers between the input and the output
- Also note the learning algorithm for the multilayer perceptron is known as back propagation.

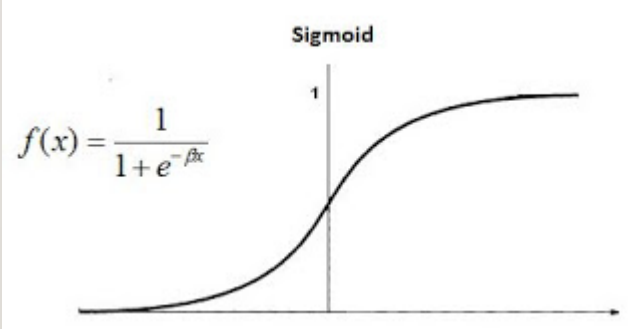
HOW THE MULTILAYER PERCEPTRON WORKS

- In MLP, the neurons use non-linear activation functions that is designed to model the behaviour of the neurons in the human brain.

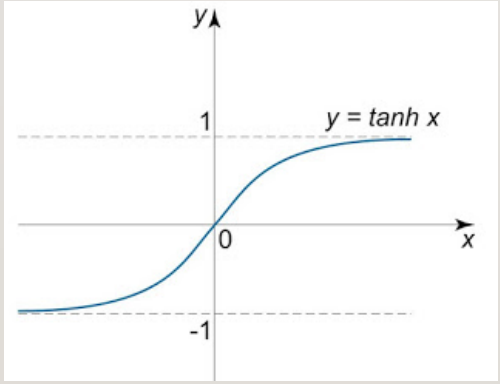
- An multi-layer perceptron has a linear activation function in all its neuron and uses backpropagation for its training.

ACTIVATION FUNCTIONS

- The activation function combines the input to the neuron with the weights and then adds a bias to produce the output.
- In other words, the activation function maps the weighted inputs to the output of the neuron.
- One of such activation functions is the sigmoid function which is used to determine the output of the neuron.
- An example of a sigmoid function is the **logistic function** which is shown as



- Another example of a sigmoid function, is the **hyperbolic tangent** activation function shown below which produces an output ranging between -1 and 1



- Yet another type of activation function that can be used is the Rectified Linear Unit or ReLU which is said to have better performance than the logistic function and the hyperbolic tangent function.

APPLYING ACTIVATION FUNCTION TO MLP

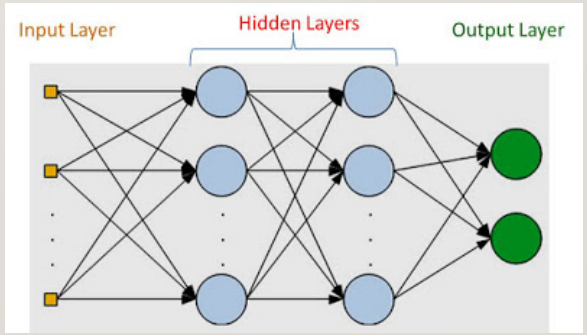
- With activation function, we can calculate the output of any neuron in the MLP.
- Assuming **w** denotes the vector of weights, **x** is the vector of inputs, *b* is the bias and ϕ is the activation function, the for the i^{th}, neuron, the output *y* would be given by:

$$y = \left(\sum_{i=1}^{n} w_i x_i + b \right) = \varphi(\mathbf{w}^T \mathbf{x} + b)$$

- An MLP is made up of a set of nodes which forms the input layer, one or more hidden layers, and an output layer.

LAYERS OF MULTILAYER PERCEPTRON(HIDDEN LAYERS)

- Remember that from the definition of multilayer perceptron, there must be one or more hidden layers.
- This means that in general, the layers of an MLP should be a minimum of three layers, since we have also the input and the output layer.
- This is illustrated in the figure below.



- Also to note is that the function activating these hidden layers has to be non-linear function (activation function) as discussed in previous section/

TRAINING/LEARNING IN MULTILAYER PERCEPTRONS

- The training process of the MLP occurs by continuous adjustment of the weights of the connections after each processing.
- This adjustment is based on the error in output(which is the different between the expected result and the output).
- This continuous adjustment of the weights is a supervised learning process called ‘backpropagation’.
- The backpropagation algorithm consists of two parts:
- forward pass
- backward pass

- In the forward pass, the expect output corresponding to the given inputs are evaluated.
- In the backward pass, partial derivatives of the cost function with respects to the different parameters are propagated back through the network.
- The process continues until the error is at the lowest value.

BACKPROPAGATION ALGORITHM

- **Backpropagation** is the essence of neural network training.
- It is the method of fine-tuning the weights of a neural network based on the error rate obtained in the previous epoch (i.e., iteration).
- Proper tuning of the weights allows us to reduce error rates and make the model reliable by increasing its generalization.
- Backpropagation in neural network is a short form for "backward propagation of errors.“
- It is a standard method of training artificial neural networks.
- This method helps calculate the gradient of a loss function with respect to all the weights in the network.

HOW BACKPROPAGATION ALGORITHM WORKS

- The Back propagation algorithm in neural network computes the gradient of the loss function for a single weight by the chain rule.
- It efficiently computes one layer at a time, unlike a native direct computation.
- It computes the gradient, but it does not define how the gradient is used. It generalizes the computation in the delta rule.

- Consider the following Back propagation neural network example diagram to understand:

❖ Inputs X, arrive through the preconnected path

❖ Input is modeled using real weights W. The weights are usually randomly selected.

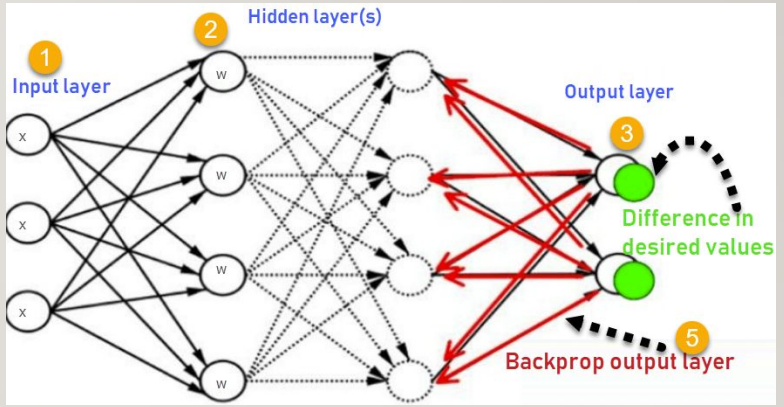
❖ Calculate the output for every neuron from the input layer, to the hidden layers, to the output layer.

❖ Calculate the error in the outputs

ErrorB= Actual Output – Desired Output

❖ Travel back from the output layer to the hidden layer to adjust the weights such that the error is decreased.

- Keep repeating the process until the desired output is achieved



FORWARD AND BACKWARD PASSES

- To train a neural network, there are 2 passes (phases):
- Forward
- Backward

- In the forward pass, we start by propagating the data inputs to the input layer, go through the hidden layer(s), measure the network's predictions from the output layer, and finally calculate the network error based on the predictions the network made.

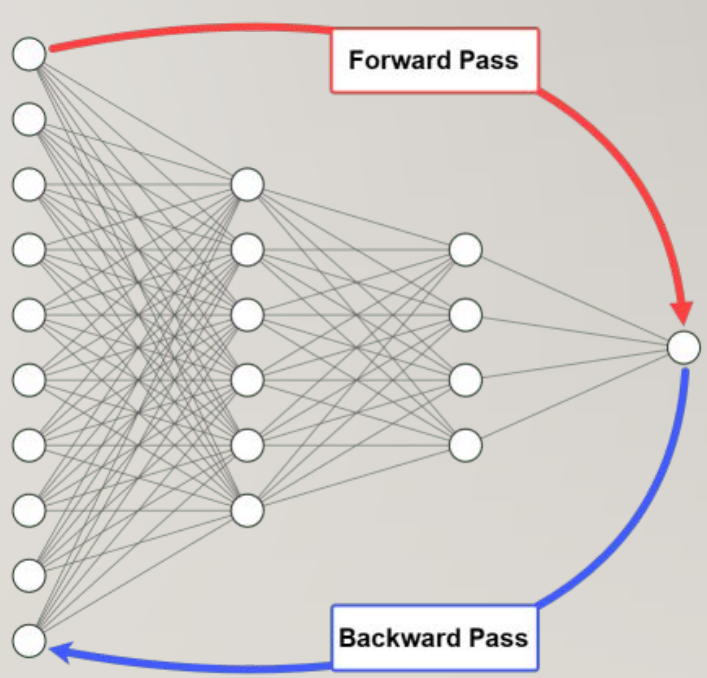
- This network error measures how far the network is from making the correct prediction.
- For example, if the correct output is 4 and the network's prediction is 1.3, then the absolute error of the network is 4-1.3=2.7.
- Note that the process of propagating the inputs from the input layer to the output layer is called **forward propagation**.
- Once the network error is calculated, then the forward propagation phase has ended, and backward pass starts.

- In the backward pass, the flow is reversed so that we start by propagating the error to the output layer until reaching the input layer passing through the hidden layer(s).

- The process of propagating the network error from the output layer to the input layer is called **backward propagation**, or simple **backpropagation**.

- The backpropagation algorithm is the set of steps used to update network weights to reduce the network error.

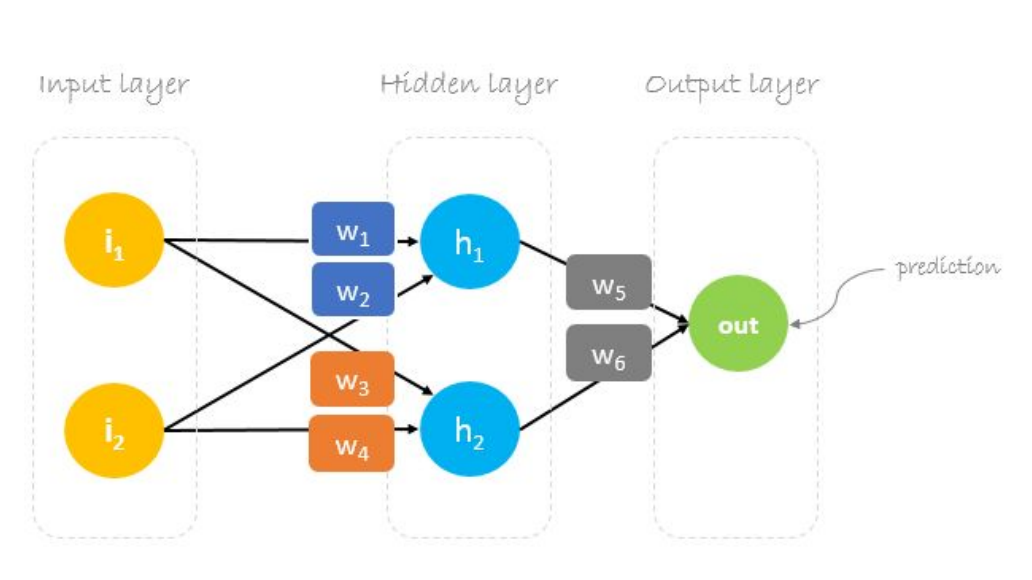
- The forward and backward phases are repeated from some epochs. In each epoch, the following occurs:



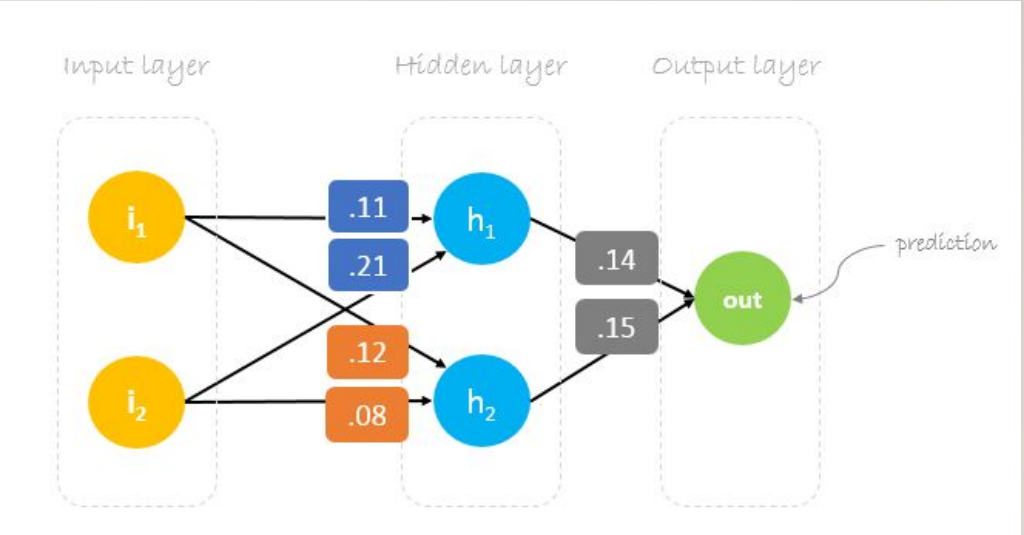
- The inputs are propagated from the input to the output layer.
- The network error is calculated.
- The error is propagated from the output layer to the input layer.

HOW BACKPROPAGATION ALGORITHM WORKS

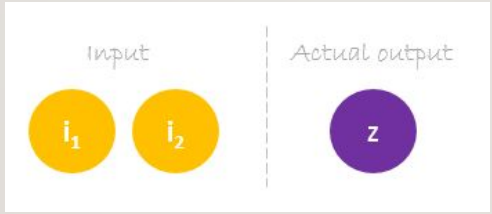
- We will build a neural network with three layers:
- **Input** layer with two inputs neurons
- One **hidden** layer with two neurons
- **Output** layer with a single neuron



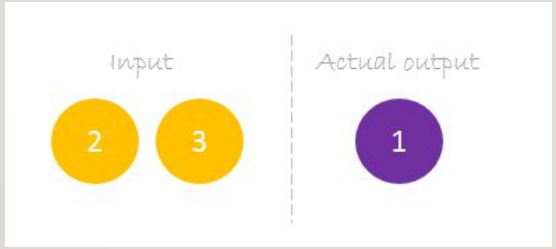
- **Weights**
- Neural network training is about finding weights that minimize prediction error.
- We usually start our training with a set of randomly generated weights.
- Then, backpropagation is used to update the weights in an attempt to correctly map arbitrary inputs to outputs.
- Our initial weights will be as following: w1 = 0.11, w2 = 0.21, w3 = 0.12, w4 = 0.08, w5 = 0.14 and w6 = 0.15



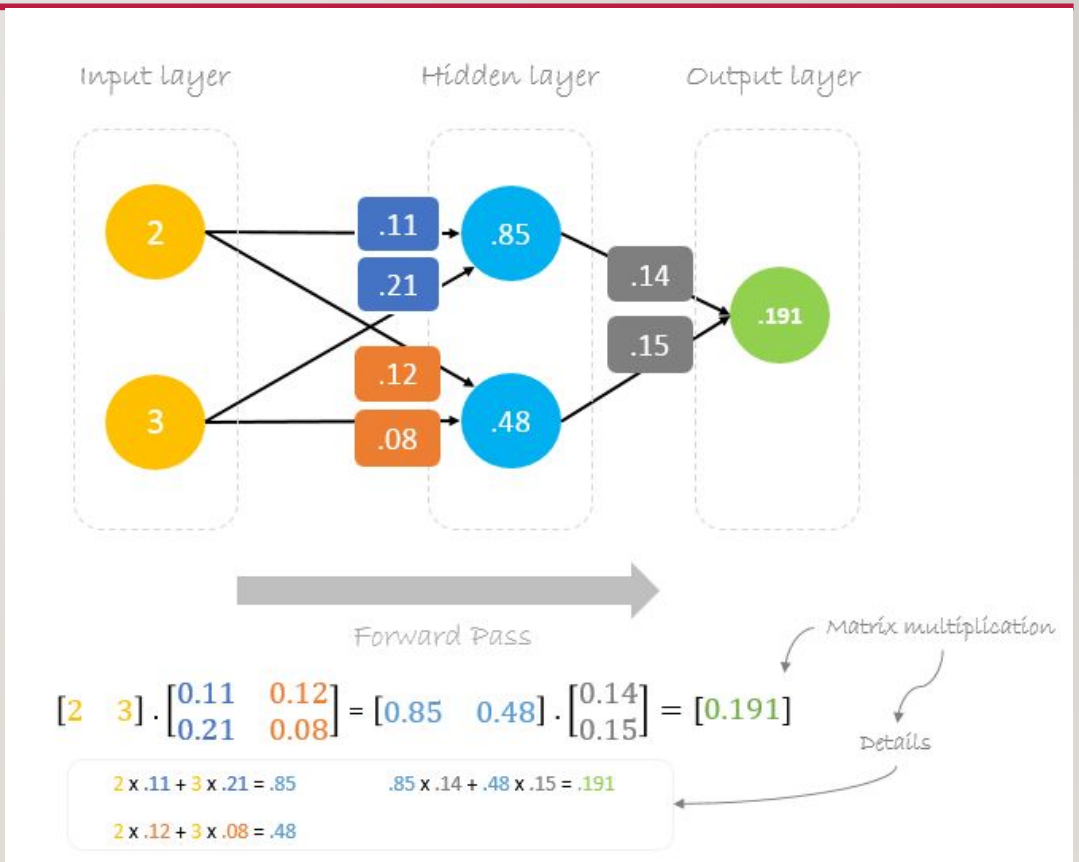
- **Dataset**
- Our dataset has one sample with two inputs and one output.



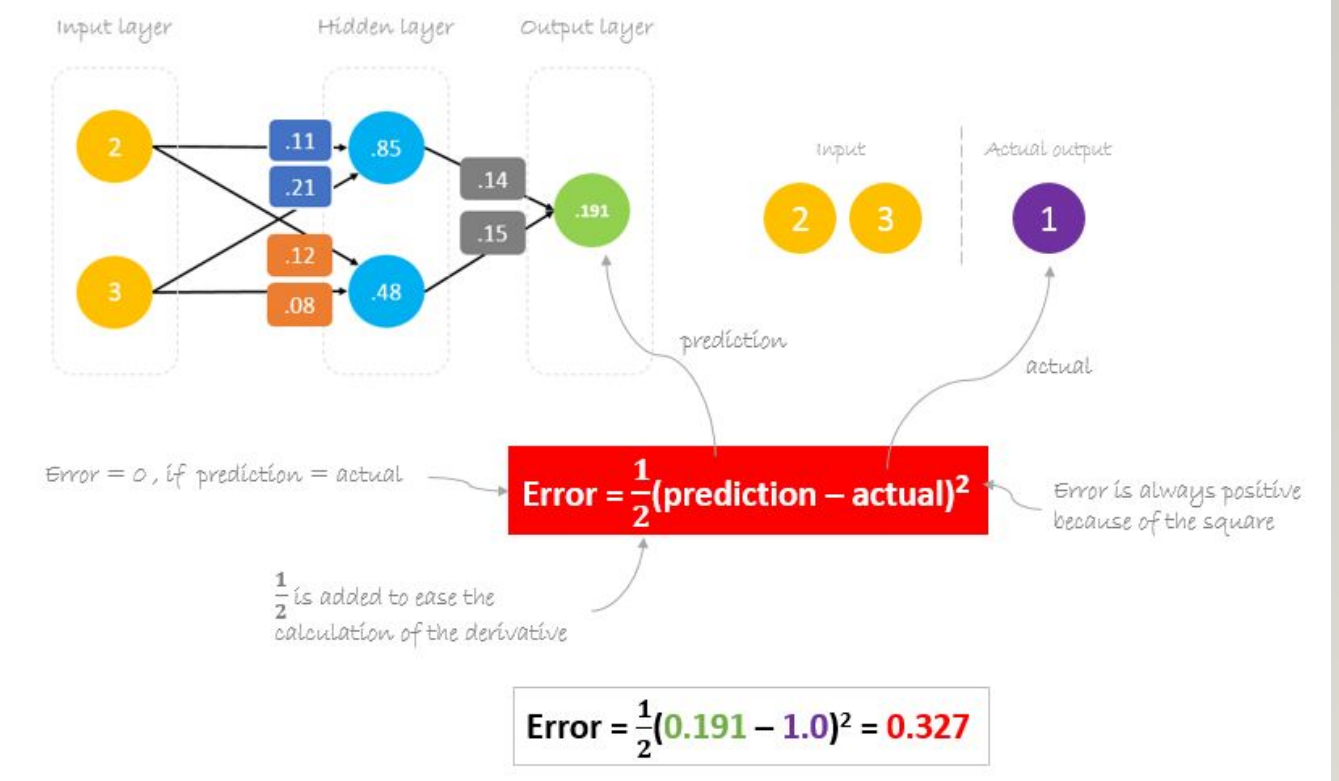
- Our single sample is as following inputs=[2, 3] and output=[1].



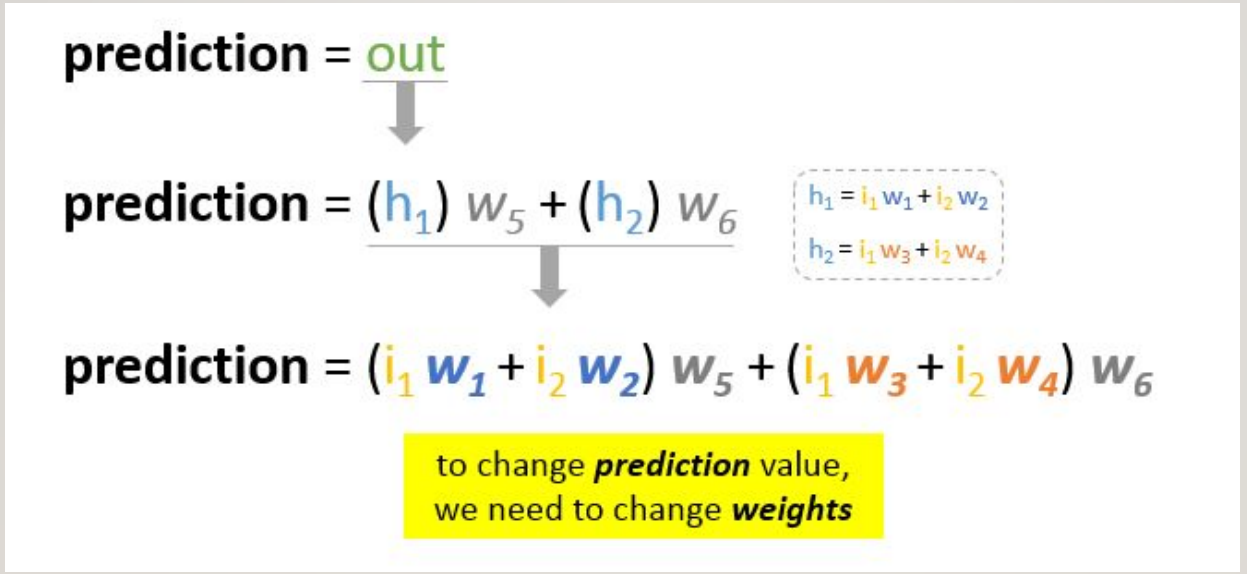
- **Forward Pass**
- We will use given weights and inputs to predict the output.
- Inputs are multiplied by weights; the results are then passed forward to next layer.



- **Calculating Error**
- Now, it's time to find out how our network performed by calculating the difference between the actual output and predicted one.
- It's clear that our network output, or **prediction**, is not even close to **actual output**.
- We can calculate the difference or the error.



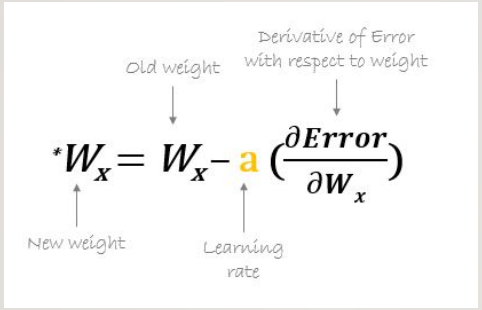
- **Reducing Error**
- Our main goal of the training is to reduce the **error** or the difference between **prediction** and **actual output**.
- Since **actual output** is constant, "not changing", the only way to reduce the error is to change **prediction** value. The question now is, how to change **prediction** value?
- By decomposing **prediction** into its basic elements we can find that **weights** are the variable elements affecting **prediction** value.
- In other words, in order to change **prediction** value, we need to change **weights** values.



- The question now is **how to change\update the weights value so that the error is reduced?**
  The answer is **Backpropagation!**

BACKPROPAGATION

- **Backpropagation**, short for "backward propagation of errors", is a mechanism used to update the **weights** using gradient descent.
- It calculates the gradient of the error function with respect to the neural network's weights.
- The calculation proceeds backwards through the network.
- **Gradient descent** is an iterative optimization algorithm for finding the minimum of a function; in our case we want to minimize th error function.
- To find a local minimum of a function using gradient descent, one takes steps proportional to the negative of the gradient of the function at the current point.



- For example, to update w6, we take the current w6 and subtract the partial derivative of error function with respect to w6.
- Optionally, we multiply the derivative of the error function by a selected number to make sure that the new updated weight is minimizing the error function; this number is called learning rate.

$$^*W_6 = W_6 - a\left(\frac{\partial Error}{\partial W_6}\right)$$

- The derivation of the error function is evaluated by applying the chain rule as following

$$\frac{\partial Error}{\partial W_6} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial W_6}$$

chain rule

$$\text{Error} = \frac{1}{2}(\text{prediction} - \text{actual})^2$$

$$\text{prediction} = (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$$

$$\frac{\partial Error}{\partial W_6} = \frac{\frac{1}{2}(predictoin - actula)^2}{\partial prediciton} * \frac{\partial \left(i_1 w_1 + i_2 w_2\right) w_5 + \left(i_1 w_3 + i_2 w_4\right) w_6}{\partial W_6}$$

$$\frac{\partial Error}{\partial W_6} = 2 * \frac{1}{2}(\text{predictoin} - \text{actula}) \frac{\partial (predictoin - actula)}{\partial prediciton} * (i_1 w_3 + i_2 w_4)$$

$h_2 = i_1 w_3 + i_2 w_4$

$$\frac{\partial Error}{\partial W_6} = (\text{predictoin} - \text{actula}) * (h_2)$$

Δ = prediction – actual

delta

$$\frac{\partial Error}{\partial W_6} = \Delta h_2$$

- So to update w6 we can apply the following formula

$$^{*}W_6 = W_6 - a\,\Delta h_2$$

- Similarly, we can derive the update formula for w5 and any other weights existing between the output and the hidden layer.

$$^{*}W_5 = W_5 - a\,\Delta h_1$$

- However, when moving backward to update w1, w2, w3 and w4 existing between input and hidden layer, the partial derivative for the error function with respect to w1, for example, will be as following.

$$\frac{\partial Error}{\partial W_1} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial h_1} * \frac{\partial h_1}{\partial W_1}$$ ← chain rule

$$\frac{\partial Error}{\partial W_1} = \frac{\partial \frac{1}{2}(predictoin - actula)^2}{\partial prediciton} * \frac{\partial (h_1)\, w_5 + (h_2)\, w_6}{\partial h_1} * \frac{\partial\, i_1 w_1 + i_2 w_2}{\partial w_1}$$

$$\frac{\partial Error}{\partial W_1} = 2 * \frac{1}{2}(\mathbf{predictoin - actula}) \frac{\partial (predictoin - actula)}{\partial prediciton} * (w_5) * (i_1)$$

$$\frac{\partial Error}{\partial W_1} = (\mathbf{predictoin - actula}) * (w_5 i_1)$$

$$\frac{\partial Error}{\partial W_1} = \Delta\, w_5 i_1$$

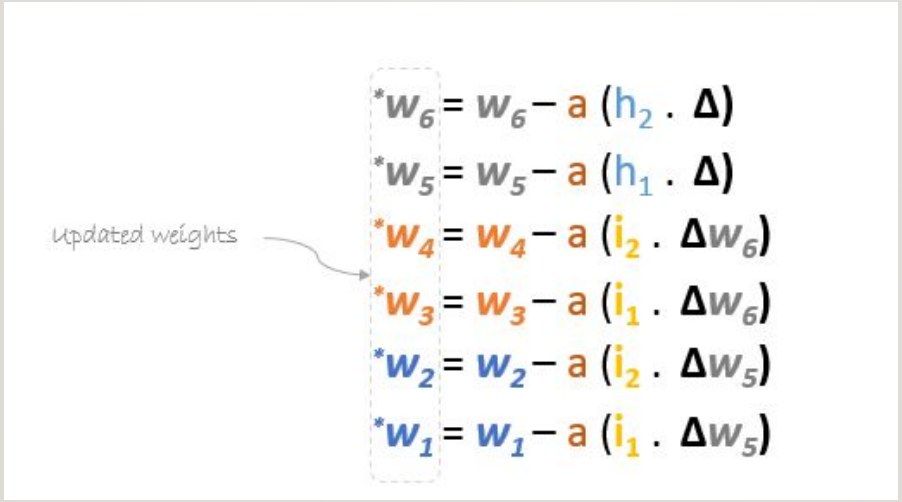
Error = $\frac{1}{2}$(prediction – actual)^2

prediction = $(h_1)\, w_5 + (h_2)\, w_6$

$h_1 = i_1 w_1 + i_2 w_2$

Δ = prediction – actual ← delta

- We can find the update formula for the remaining weights w2, w3 and w4 in the same way.
- In summary, the update formulas for all weights will be as following:



- We can rewrite the update formulas in matrices as following

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a\,\Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} a h_1 \Delta \\ a h_2 \Delta \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a\,\Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} . [w_5 \quad w_6] = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} a\, i_1 \Delta w_5 & a\, i_1 \Delta w_6 \\ a\, i_2 \Delta w_5 & a\, i_2 \Delta w_6 \end{bmatrix}$$

- **Backward Pass**
- Using derived formulas we can find the new weights.
- Learning rate: is a hyperparameter which means that we need to manually guess its value.

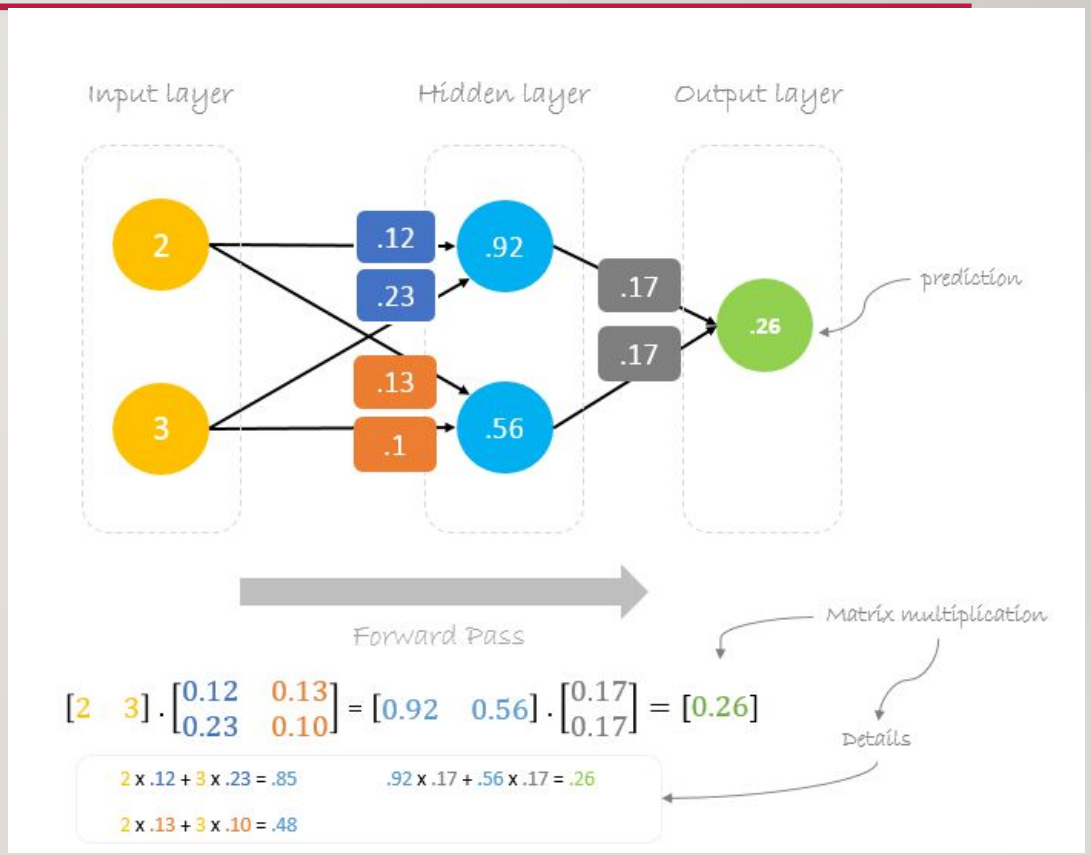
$\Delta = 0.191 - 1 = -0.809$ ← **Delta** = prediction - actual

$a = 0.05$ ← **Learning rate,** we smartly guess this number

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - 0.05(-0.809)\begin{bmatrix} 0.85 \\ 0.48 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - \begin{bmatrix} -0.034 \\ -0.019 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - 0.05(-0.809)\begin{bmatrix} 2 \\ 3 \end{bmatrix} . \begin{bmatrix} 0.14 & 0.15 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - \begin{bmatrix} -0.011 & -0.012 \\ -0.017 & -0.018 \end{bmatrix} = \begin{bmatrix} .12 & .13 \\ .23 & .10 \end{bmatrix}$$

- Now, using the new **weights** we will repeat the forward passed
- We can notice that the prediction 0.26 is a little bit closer to actual output than the previously predicted one 0.191.
- We can repeat the same process of backward and forward pass until error is close or equal to zero.



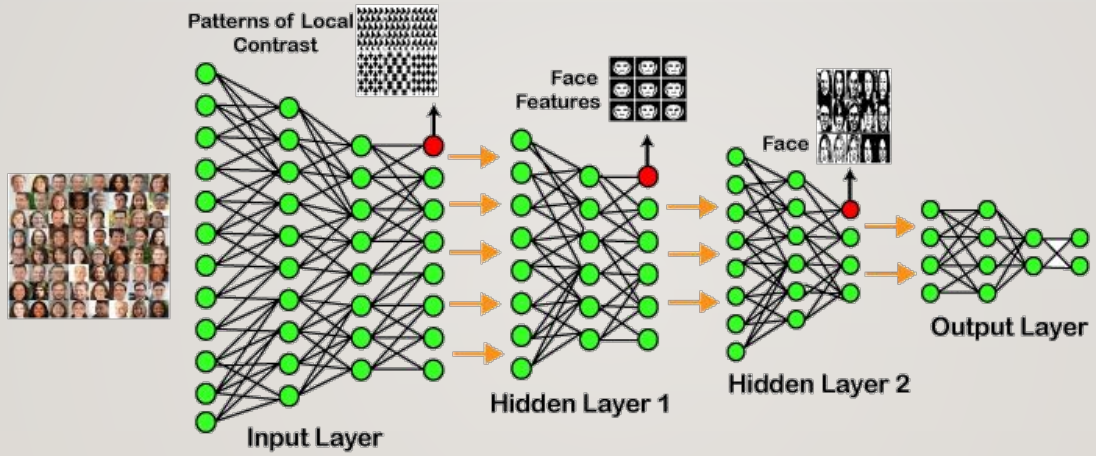
DEEP LEARNING

- Deep learning is based on the branch of machine learning, which is a subset of artificial intelligence.
- Since neural networks imitate the human brain and so deep learning will do.
- In deep learning, nothing is programmed explicitly.
- Basically, it is a machine learning class that makes use of numerous nonlinear processing units so as to perform feature extraction as well as transformation.
- The output from each preceding layer is taken as input by each one of the successive layers.

- Deep learning models are capable enough to focus on the accurate features themselves by requiring a little guidance from the programmer and are very helpful in solving out the problem of dimensionality.
- Deep learning algorithms are used, especially when we have a huge no of inputs and outputs.
- Since deep learning has been evolved by the machine learning, which itself is a subset of artificial intelligence and as the idea behind the artificial intelligence is to mimic the human behavior, so same is "the idea of deep learning to build such algorithm that can mimic the brain".

- Deep learning is implemented with the help of Neural Networks, and the idea behind the motivation of Neural Network is the biological neurons, which is nothing but a brain cell.
- **Deep learning is a collection of statistical techniques of machine learning for learning feature hierarchies that are actually based on artificial neural networks.**
- So basically, deep learning is implemented by the help of deep networks, which are nothing but neural networks with multiple hidden layers.

EXAMPLE OF DEEP LEARNING



- In the example given above, we provide the raw data of images to the first layer of the input layer.
- After then, these input layer will determine the patterns of local contrast that means it will differentiate on the basis of colors, luminosity, etc.

- Then the 1st hidden layer will determine the face feature, i.e., it will fixate on eyes, nose, and lips, etc.
- And then, it will fixate those face features on the correct face template.
- So, in the 2nd hidden layer, it will actually determine the correct face here as it can be seen in the above image, after which it will be sent to the output layer.
- Likewise, more hidden layers can be added to solve more complex problems, for example, if we want to find out a particular kind of face having large or light complexions.
- So, as and when the hidden layers increase, we are able to solve complex problems.

ARCHITECTURES

- ***Deep Neural Networks***
- It is a neural network that incorporates the complexity of a certain level, which means several numbers of hidden layers are encompassed in between the input and output layers. They are highly proficient on model and process non-linear associations.
- ***Deep Belief Networks***
- A deep belief network is a class of Deep Neural Network that comprises of multi-layer belief networks.
- **Steps to perform DBN:**
  - With the help of the Contrastive Divergence algorithm, a layer of features is learned from perceptible units.
  - Next, the formerly trained features are treated as visible units, which perform learning of features.
  - Lastly, when the learning of the final hidden layer is accomplished, then the whole DBN is trained.

- ***Recurrent Neural Networks***
- It permits parallel as well as sequential computation, and it is exactly similar to that of the human brain (large feedback network of connected neurons).
- Since they are capable enough to reminisce all of the imperative things related to the input they have received, so they are more precise

TYPES OF DEEP LEARNING NETWORKS

- **1. Feed Forward Neural Network**
- A feed-forward neural network is none other than an Artificial Neural Network, which ensures that the nodes do not form a cycle.
- In this kind of neural network, all the perceptrons are organized within layers, such that the input layer takes the input, and the output layer generates the output.
- Since the hidden layers do not link with the outside world, it is named as hidden layers.
- Each of the perceptrons contained in one single layer is associated with each node in the subsequent layer.

- It can be concluded that all of the nodes are fully connected. It does not contain any visible or invisible connection between the nodes in the same layer.
- There are no back-loops in the feed-forward network. To minimize the prediction error, the backpropagation algorithm can be used to update the weight values.
- **Applications:**
- Data Compression
- Pattern Recognition
- Computer Vision
- Sonar Target Recognition
- Speech Recognition
- Handwritten Characters Recognition

- **2. Recurrent Neural Network**
- Recurrent neural networks are yet another variation of feed-forward networks.
- Here each of the neurons present in the hidden layers receives an input with a specific delay in time.
- The Recurrent neural network mainly accesses the preceding info of existing iterations.
- For example, to guess the succeeding word in any sentence, one must have knowledge about the words that were previously used.
- It not only processes the inputs but also shares the length as well as weights crossways time.

- It does not let the size of the model to increase with the increase in the input size.
- However, the only problem with this recurrent neural network is that it has slow computational speed as well as it does not contemplate any future input for the current state.
- It has a problem with reminiscing prior information.
- **Applications:**
  - Machine Translation
  - Robot Control
  - Time Series Prediction
  - Speech Recognition
  - Speech Synthesis
  - Time Series Anomaly Detection
  - Rhythm Learning
  - Music Composition

- **3. Convolutional Neural Network**
- Convolutional Neural Networks are a special kind of neural network mainly used for image classification, clustering of images and object recognition.
- CNNs enable unsupervised construction of hierarchical image representations. To achieve the best accuracy, deep convolutional neural networks are preferred more than any other neural network.
- **Applications:**

•Identify Faces, Street Signs, Tumors.
•Image Recognition.
•Video Analysis.
•NLP.
•Anomaly Detection.
•Drug Discovery.
•Checkers Game.
•Time Series Forecasting.

- **4. Restricted Boltzmann Machine**
- RBMs are yet another variant of Boltzmann Machines.
- Here the neurons present in the input layer and the hidden layer encompasses symmetric connections amid them.
- However, there is no internal association within the respective layer.
- But in contrast to RBM, Boltzmann machines do encompass internal connections inside the hidden layer. These restrictions in BMs helps the model to train efficiently.
- **Applications:**
- Filtering.
- Feature Learning.
- Classification.
- Risk Detection.
- Business and Economic analysis.

- **5. Autoencoders**
- An autoencoder neural network is another kind of unsupervised machine learning algorithm.
- Here the number of hidden cells is merely small than that of the input cells.
- But the number of input cells is equivalent to the number of output cells.
- An autoencoder network is trained to display the output similar to the fed input to force AEs to find common patterns and generalize the data.
- The autoencoders are mainly used for the smaller representation of the input. It helps in the reconstruction of the original data from compressed data. This algorithm is comparatively simple as it only necessitates the output identical to the input.
- **Encoder:** Convert input data in lower dimensions.
- **Decoder:** Reconstruct the compressed data.
- **Applications:**
  - Classification.
  - Clustering.
  - Feature Compression.

DEEP LEARNING APPLICATIONS

- ***Self-Driving Cars***
- In self-driven cars, it is able to capture the images around it by processing a huge amount of data, and then it will decide which actions should be incorporated to take a left or right or should it stop. So, accordingly, it will decide what actions it should take, which will further reduce the accidents that happen every year.
- ***Voice Controlled Assistance***
- When we talk about voice control assistance, then **Siri** is the one thing that comes into our mind. So, you can tell Siri whatever you want it to do it for you, and it will search it for you and display it for you.
- ***Automatic Image Caption Generation***
- Whatever image that you upload, the algorithm will work in such a way that it will generate caption accordingly. If you say blue colored eye, it will display a blue-colored eye with a caption at the bottom of the image.
- ***Automatic Machine Translation***
- With the help of automatic machine translation, we are able to convert one language into another with the help of deep learning.